Reto Técnico Full Stack

Java Quarkus / Angular / Base de Datos Relacional

Información General

- ### Enunciado del Problema

Una institución financiera requiere un **Sistema de Gestión de Transferencias Interbancarias**. El sistema debe permitir la administración de cuentas bancarias, la ejecución de transferencias entre cuentas con validaciones de negocio, la detección automática de operaciones sospechosas y la consulta del historial de movimientos.

El backend debe implementarse como una API REST con Java Quarkus, persistiendo la información en una base de datos relacional (SQL Server, MySQL u Oracle). El frontend, desarrollado en Angular, debe ofrecer un dashboard operativo para visualizar cuentas y ejecutar transferencias. La API debe estar documentada con Swagger/OpenAPI.

- **Reglas de negocio clave:**
  1. Cada cuenta tiene un número único de 10 dígitos, un titular, un tipo (ahorro o corriente), un saldo y un estado (activa o inactiva).
  2. Una transferencia solo procede si ambas cuentas existen, están activas y la cuenta origen tiene saldo suficiente.
  3. Una operación se marca como "sospechosa" si el monto supera $5,000 o si la cuenta origen acumula más de 5 transferencias en la última hora.
  4. Ninguna cuenta puede transferir más de $10,000 acumulados en un día calendario.
  5. Las transferencias deben ser atómicas: se actualizan ambos saldos en una sola transacción de base de datos, o no se actualiza ninguno.

- ### Stack Tecnológico Recomendado

Capa	Tecnología
Backend	Java 17+ con Quarkus (RESTEasy, Hibernate ORM con Panache)
Frontend	Angular 14+ (TypeScript, Angular Material o PrimeNG opcional)
Base de Datos	SQL Server, MySQL u Oracle (a elección del candidato)
Documentación API	Swagger / OpenAPI
Control de versiones	Git + repositorio público en GitHub

Nombre del proyecto: EvaluacionNombreApellido

(ejemplo: EvaluacionCarlosPerez)

1. Criterios de Aceptación - Evaluación del Backend

➢ CA-01: Registro y gestión de cuentas bancarias

Criterio: El sistema debe exponer un endpoint REST (`POST /api/cuentas`) que permita registrar cuentas bancarias con los campos: número de cuenta (único, 10 dígitos), titular, tipo de cuenta (ahorro/corriente), saldo inicial (≥ 0) y estado (activa/inactiva). Debe validar que no existan duplicados por número de cuenta y retornar la cuenta creada con su identificador. Adicionalmente, exponer un endpoint:

- **`GET /api/cuentas`** para listar todas las cuentas registradas.

Archivos esperados: `CuentaResource.java` o `CuentaController.java`, `Cuenta.java` (entidad JPA), `CuentaRepository.java` o `CuentaService.java`.

Evaluación:

- **Sí:** El endpoint recibe los campos indicados, valida duplicados y saldo negativo, persiste en base de datos y retorna la cuenta creada con código HTTP 201. El endpoint GET lista las cuentas correctamente.
- **Parcial:** El endpoint existe pero le faltan validaciones (no verifica duplicados o acepta saldo negativo), o no retorna el código HTTP adecuado.
- **No:** No existe endpoint de registro o no persiste en base de datos.

➢ CA-02: Transferencias entre cuentas con validaciones de negocio

Criterio: El sistema debe exponer un endpoint REST (`POST /api/transferencias`) que ejecute transferencias entre cuentas. Debe validar: que ambas cuentas existan y estén activas, que la cuenta origen tenga saldo suficiente y que el monto sea mayor a cero. La operación debe ser **atómica** (actualizar saldos de ambas cuentas en una sola transacción de base de datos usando `@Transactional`). En caso de error, retornar un mensaje descriptivo con el código HTTP apropiado (400, 404, 422).

Archivos esperados: `TransferenciaResource.java`, `TransferenciaService.java`, `Transferencia.java`

Evaluación:

- **Sí:** El endpoint ejecuta transferencias con todas las validaciones, la operación es atómica (`@Transactional`), actualiza saldos correctamente y maneja errores con códigos HTTP apropiados.
- **Parcial:** El endpoint funciona pero le faltan validaciones (no verifica estado de cuentas o saldo), o la operación no es atómica, o los códigos de error no son descriptivos.
- **No:** No existe endpoint de transferencias o no actualiza saldos.

➢ CA-03: Detección de operaciones sospechosas

Criterio: Implementar un servicio que marque automáticamente una transferencia como "requiere revisión" cuando se cumpla alguna de estas condiciones: (a) el monto individual supera $5,000, o (b) la cuenta origen ha realizado más de 5 transferencias en la última hora. El servicio debe invocarse automáticamente durante el proceso de transferencia y registrar el estado correspondiente en la entidad.

Archivos esperados: `DeteccionSospechosaService.java` o lógica integrada en `TransferenciaService.java`.

Evaluación:

- **Sí:** El servicio detecta ambas condiciones, se invoca automáticamente al crear una transferencia y marca el estado correctamente en la base de datos.
- **Parcial:** Detecta solo una de las dos condiciones, o la detección existe pero no se invoca automáticamente durante la transferencia.
- **No:** No existe lógica de detección de operaciones sospechosas.

➢ CA-04: Control de límite diario de transferencias

Criterio: Implementar un servicio que controle que ninguna cuenta pueda transferir más de $10,000 acumulados en un día calendario. Si una transferencia haría que el acumulado del día supere ese límite, debe ser rechazada con un mensaje informativo que indique el monto restante disponible.

Archivos esperados: Lógica en `TransferenciaService.java` o servicio dedicado `LimiteDiarioService.java`.

Evaluación:

- **Sí:** El servicio calcula correctamente el acumulado diario, rechaza transferencias que excedan el límite e informa el monto restante disponible.
- **Parcial:** Implementa el control pero no calcula correctamente el acumulado diario, o no informa el monto restante.
- **No:** No existe control de límite diario.

➢ CA-05: Consulta de historial y resumen de cuenta

Criterio: El sistema debe exponer: (a) un endpoint `GET /api/transferencias` que permita consultar el historial de transferencias con filtros opcionales por cuenta origen, cuenta destino, rango de fechas y estado (completada/rechazada/en revisión), retornando resultados ordenados por fecha descendente; y (b) un endpoint `GET /api/cuentas/{id}/resumen` que retorne un resumen con: saldo actual, total transferido hoy, cantidad de transferencias del día y cantidad marcadas como sospechosas.

Archivos esperados: Endpoints en `TransferenciaResource.java` y `CuentaResource.java`, queries en repositorios o servicios.

Evaluación:

- **Sí:** Ambos endpoints existen, los filtros funcionan correctamente, el resumen calcula datos en tiempo real y los resultados están ordenados por fecha descendente.
- **Parcial:** Solo uno de los dos endpoints está implementado, o los filtros no funcionan correctamente, o el resumen tiene cálculos incorrectos.
- **No:** No existen endpoints de consulta.

2. Criterios de Aceptación - Evaluación del Frontend

➢ CA-06: Dashboard de cuentas bancarias

Criterio: Desarrollar un componente Angular que presente una tabla con las cuentas bancarias registradas, mostrando: número de cuenta, titular, tipo de cuenta, saldo actual y estado. La tabla debe permitir ordenamiento por saldo (ascendente/descendente) y consumir el endpoint `GET /api/cuentas` del backend.

Archivos esperados: `cuentas.component.ts`, `cuentas.component.html`, `cuenta.service.ts`.

Evaluación:

- **Sí:** El componente muestra todas las cuentas con los campos indicados, consume el endpoint del backend correctamente, y el ordenamiento por saldo funciona.
- **Parcial:** El componente existe pero no consume el backend (usa datos mock) o el ordenamiento no funciona.
- **No:** No existe componente de listado de cuentas.

➢ CA-07: Formulario de transferencia

Criterio: Implementar un formulario Angular para ejecutar nuevas transferencias. Debe incluir: selección de cuenta origen, selección de cuenta destino, monto y un botón de envío. El formulario debe validar en el cliente que: todos los campos sean requeridos, el monto sea numérico y mayor a cero, y la cuenta origen sea diferente a la destino. Debe mostrar mensajes de éxito o error provenientes del backend.

Archivos esperados: `transferencia-form.component.ts`, `transferencia-form.component.html`, `transferencia.service.ts`.

Evaluación:

- **Sí:** El formulario tiene todos los campos, las validaciones del cliente funcionan, consume el endpoint POST del backend y muestra mensajes de éxito/error.
- **Parcial:** El formulario existe pero le faltan validaciones del lado del cliente, o no muestra mensajes de respuesta del backend.
- **No:** No existe formulario de transferencia.

3. Criterios de Aceptación - Requisitos y Entregables

➢ CA-08: Arquitectura y separación de responsabilidades

Criterio: La arquitectura del código fuente debe seguir un enfoque en capas con separación clara de responsabilidades: Resources/Controllers, Services, Repositories. La lógica de

negocio no debe residir en controladores ni repositorios. El código debe demostrar modularidad y escalabilidad.

Evaluación:

- **Sí:** La estructura del proyecto tiene separación clara en capas, la lógica de negocio está en servicios, los controladores solo delegan, y los repositorios solo acceden a datos. No hay code smells significativos.
- **Parcial:** Existe una estructura de capas, pero la separación no es estricta (lógica de negocio en controladores, queries complejas en repositorios sin justificación), o existen múltiples code smells.
- **No:** No hay separación de capas, todo el código está en un solo archivo o capa.

➢ CA-09: Calidad del código y buenas prácticas

Criterio: El código debe seguir buenas prácticas y estándares de codificación: nombrado descriptivo en inglés o español (consistente), principios SOLID, manejo centralizado de excepciones (ExceptionMapper en Quarkus), uso adecuado de inyección de dependencias (`@Inject`), uso de DTOs para requests/responses (no exponer entidades JPA directamente), y anotaciones JPA/Hibernate correctas.

Evaluación:

- **Sí:** El código sigue las prácticas descritas, tiene un ExceptionMapper centralizado, usa DTOs, inyección de dependencias y el nombrado es consistente y descriptivo.
- **Parcial:** Cumple algunas prácticas pero no todas — por ejemplo, no usa DTOs y expone entidades directamente, o el manejo de excepciones no está centralizado.
- **No:** El código no sigue estándares, tiene nombrado inconsistente, no maneja excepciones y expone entidades directamente.

➢ CA-10: Documentación (Swagger + README)

Criterio: La API debe estar documentada con Swagger/OpenAPI incluyendo: descripción de cada endpoint, parámetros de entrada/salida, códigos de respuesta posibles y ejemplos. El repositorio debe incluir un archivo `README.md` con instrucciones claras para: configuración de la base de datos (script de creación o instrucciones), compilación del proyecto backend (`mvn` o `gradle`), ejecución del backend y frontend, y URL de acceso a la documentación Swagger.

Evaluación:

- **Sí:** Swagger está configurado con descripciones completas de endpoints. El README tiene instrucciones claras y suficientes para compilar y ejecutar el proyecto.
- **Parcial:** Swagger existe pero sin descripciones, o el README es incompleto (falta configuración de BD o pasos de ejecución).
- **No:** No hay documentación Swagger ni README.

4. Extras - Bonificación (No Obligatorios)

Estos ítems no son requeridos para aprobar la prueba, pero suman puntos adicionales y demuestran nivel senior:

- **Docker / Docker Compose:** Archivo docker-compose.yml que levante el entorno completo (backend + base de datos + frontend) con un solo comando.
- **Pruebas unitarias:** Tests con JUnit 5 y Mockito para los servicios de negocio (detección sospechosa, límite diario, transferencias).
- **Pruebas de integración:** Tests con @QuarkusTest que validen los endpoints contra la base de datos.
- **Paginación:** Implementar paginación en los endpoints de listado (GET /api/cuentas, GET /api/transferencias).
- **DTOs y Mappers:** Uso de MapStruct o mapeo manual con DTOs de request/response separados de las entidades JPA.
- **Logs estructurados:** Logging con @Slf4j o java.util.logging en operaciones críticas (transferencias, detección de fraude).
- **Manejo de CORS:** Configuración de CORS para permitir consumo desde el frontend Angular.

Cada extra implementado correctamente suma hasta 2 puntos adicionales sobre 100.